

DATABASE FUNDAMENTALS

A Complete Beginner's Guide

- | | |
|--------------------------|--------------------------|
| ■ What is a Database? | ■ Functions & Operations |
| ■ ■ Installation & Setup | ■ SQL Queries in Depth |
| ■ ■ Database Design | ■ Security & Permissions |
| ■ Data Types & Schemas | ■ Performance & Indexing |
| ■ ■ Backup & Recovery | ■ Best Practices & Tips |

Covers SQL, MySQL, PostgreSQL & SQLite

Chapter 1 — What is a Database?

Understanding the Concept

A **database** is an organised collection of structured information or data, typically stored electronically in a computer system. Think of it as a highly intelligent digital filing cabinet that can store millions of records, retrieve them in milliseconds, and keep them consistent and safe.

Databases are the backbone of almost every digital system you use — from the app that recommends movies, to the bank that stores your transactions, to the e-commerce site that tracks your orders.

■ *Real-world analogy: A database is like a school register. Each student is a row, each column holds specific info (name, age, grade). You can search, sort, and update easily.*

Types of Databases

Type	Description	Examples
Relational (SQL)	Data stored in tables with rows & columns; uses SQL	MySQL, PostgreSQL, SQLite, Oracle
NoSQL (Document)	Schema-free JSON-like documents	MongoDB, CouchDB, Firestore
NoSQL (Key-Value)	Simple key → value pairs; very fast lookups	Redis, DynamoDB, Memcached
NoSQL (Column)	Stores data in columns; good for analytics	Cassandra, HBase, BigQuery
NoSQL (Graph)	Nodes & edges; models relationships naturally	Neo4j, Amazon Neptune
In-Memory DB	Data lives in RAM; ultra-low latency	Redis, VoltDB, SAP HANA
Time-Series DB	Optimised for timestamped data streams	InfluxDB, TimescaleDB

Why Use a Database Instead of a Spreadsheet?

- Scale:** Databases handle millions of records; spreadsheets struggle beyond ~100,000 rows.

- **Concurrency:** Multiple users can read/write simultaneously without corruption.
- **Integrity:** Constraints ensure no invalid data ever enters the system.
- **Security:** Fine-grained access control (who can read, write, delete).
- **Speed:** Indexed queries return results in milliseconds regardless of data size.
- **Automation:** Triggers, procedures, and scheduled jobs automate complex workflows.

Chapter 2 — Installation & Setup

Choosing Your First Database

For beginners, **SQLite** is perfect — it requires zero configuration and runs entirely from a single file. For learning full server-based databases, **MySQL** or **PostgreSQL** are the industry standard choices.

SQLite (Zero-config, file-based)

```
# Pre-installed on Python. Just import:
import sqlite3
conn = sqlite3.connect('mydb.db') # creates the file
cursor = conn.cursor()
```

■ No installation needed! Perfect to learn SQL immediately.

MySQL (Linux/macOS)

```
sudo apt update && sudo apt install mysql-server # Ubuntu/Debian
brew install mysql # macOS with Homebrew
sudo mysql_secure_installation # Run the security wizard
sudo systemctl start mysql # Start the service
```

■ Windows: Download the MySQL Installer from mysql.com/downloads

PostgreSQL (Linux/macOS)

```
sudo apt install postgresql postgresql-contrib # Ubuntu/Debian
brew install postgresql # macOS
sudo systemctl start postgresql # Start service
sudo -u postgres psql # Enter the shell
```

■ Windows: Use the EDB installer at enterprisedb.com/downloads

Recommended GUI Tools

Tool	Works With	Best For	Cost
DBeaver	All databases	Universal client, great for beginners	Free
TablePlus	All databases	Beautiful modern UI	Freemium

Tool	Works With	Best For	Cost
pgAdmin 4	PostgreSQL	Full Postgres administration	Free
MySQL Workbench	MySQL	Official MySQL GUI client	Free
DB Browser	SQLite	Quick SQLite file inspection	Free
DataGrip	All databases	Professional IDE for SQL	Paid

Chapter 3 — Core Concepts & Database Design

Key Terminology

Term	Meaning
Table (Relation)	A grid of rows and columns, like a spreadsheet tab.
Row (Record/Tuple)	One single data entry in a table.
Column (Field)	A named attribute shared by all rows (e.g., 'email').
Primary Key (PK)	A unique identifier for each row (e.g., user_id).
Foreign Key (FK)	A column referencing a Primary Key in another table.
Index	A data structure that speeds up lookups on a column.
Schema	The blueprint: defines tables, columns, types, constraints.
Query	A command to retrieve, insert, update, or delete data.
Transaction	A group of operations that all succeed or all fail together.
Constraint	A rule enforced on a column (NOT NULL, UNIQUE, CHECK...).
View	A virtual table defined by a saved SELECT query.
Stored Procedure	Pre-compiled SQL logic stored in the database itself.

The ACID Properties — The Pillars of Reliability

A	Atomicity	All operations in a transaction succeed, or none of them do. No partial updates.
C	Consistency	The database always moves from one valid state to another. Constraints are never violated.

I	Isolation	Concurrent transactions don't interfere with each other. Each sees a clean snapshot.
D	Durability	Once committed, data survives crashes, power failures, and system restarts.

Normalisation — Eliminating Redundancy

Normalisation is the process of structuring a database to reduce data redundancy and improve data integrity. There are progressive **Normal Forms (NF)**:

1NF — First Normal Form: Each column holds atomic (indivisible) values. No repeating groups or arrays in a cell.

2NF — Second Normal Form: Must be in 1NF. Every non-key column is fully dependent on the entire primary key (no partial dependencies).

3NF — Third Normal Form: Must be in 2NF. No transitive dependencies — non-key columns must depend only on the primary key.

BCNF — Boyce-Codd Normal Form: A stricter version of 3NF. For every functional dependency $X \rightarrow Y$, X must be a super-key.

■ *Rule of thumb: Follow 3NF for most applications. Over-normalisation can hurt performance; sometimes intentional denormalisation is valid in read-heavy systems.*

Chapter 4 — SQL: The Language of Databases

SQL (Structured Query Language) is the standard language for interacting with relational databases. It is divided into sub-languages based on what they do:

Sub-language	Commands	Purpose
DDL — Data Definition	CREATE, ALTER, DROP, TRUNCATE	Define/modify the structure (schema)
DML — Data Manipulation	INSERT, UPDATE, DELETE, MERGE	Add, modify, remove data
DQL — Data Query	SELECT	Retrieve and filter data
DCL — Data Control	GRANT, REVOKE	Manage permissions & access
TCL — Transaction Control	BEGIN, COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

Creating a Database and Tables (DDL)


```
-- 1. Create a new database
CREATE DATABASE school_db;
USE school_db; -- (MySQL) or \c school_db (PostgreSQL)

-- 2. Create a Students table
CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  date_of_birth DATE,
  grade CHAR(1) CHECK (grade IN ('A','B','C','D','F')),
  enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Create a Courses table (related table)
CREATE TABLE courses (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  course_name VARCHAR(100) NOT NULL,
  credits TINYINT CHECK (credits BETWEEN 1 AND 6)
);

-- 4. Enrolment table (many-to-many relationship)
CREATE TABLE enrolments (
  enrolment_id INT PRIMARY KEY AUTO_INCREMENT,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
  FOREIGN KEY (course_id) REFERENCES courses(course_id) ON DELETE CASCADE
);
```

Inserting, Updating & Deleting Data (DML)

```
-- INSERT single row
INSERT INTO students (first_name, last_name, email, date_of_birth, grade)
VALUES ('Aarav', 'Sharma', 'aarav@email.com', '2002-05-14', 'A');

-- INSERT multiple rows at once
INSERT INTO students (first_name, last_name, email, grade) VALUES
('Priya', 'Patel', 'priya@email.com', 'B'),
('Rohan', 'Verma', 'rohan@email.com', 'A'),
('Sunita', 'Das', 'sunita@email.com', 'C');

-- UPDATE – always use WHERE or you'll update ALL rows!
UPDATE students SET grade = 'A' WHERE student_id = 3;

-- DELETE – same warning: use WHERE!
DELETE FROM students WHERE student_id = 5;

-- Safe delete: check first with SELECT
SELECT * FROM students WHERE student_id = 5;
```

■ **DANGER:** Never run **UPDATE** or **DELETE** without a **WHERE** clause in production. Always preview with **SELECT** first. Use transactions so you can **ROLLBACK** if needed.

SELECT Queries — Retrieving Data (DQL)

```
-- Basic SELECT
SELECT * FROM students; -- All columns, all rows
SELECT first_name, email FROM students; -- Specific columns

-- Filtering with WHERE
SELECT * FROM students WHERE grade = 'A';
SELECT * FROM students WHERE grade IN ('A', 'B') AND last_name LIKE 'S%';

-- Sorting & Limiting
SELECT * FROM students ORDER BY last_name ASC LIMIT 10;
SELECT * FROM students ORDER BY enrolled_at DESC LIMIT 5 OFFSET 10;

-- Aggregate functions
SELECT COUNT(*) AS total_students FROM students;
SELECT grade, COUNT(*) AS count FROM students GROUP BY grade;
SELECT grade, COUNT(*) AS count FROM students GROUP BY grade HAVING count > 2;

-- JOIN (combine related tables)
SELECT s.first_name, s.last_name, c.course_name
FROM enrolments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE c.credits >= 3
ORDER BY s.last_name;

-- Subquery example
SELECT * FROM students
WHERE student_id IN (SELECT student_id FROM enrolments WHERE course_id = 2);
```

Chapter 5 — Types of JOINS Explained

JOIN Type	Returns	Use When...
INNER JOIN	Only rows that match in BOTH tables	You want only complete matches
LEFT (OUTER) JOIN	All rows from left table + matched rows from right	You want all left-side rows even if no match
RIGHT (OUTER) JOIN	All rows from right table + matched rows from left	You want all right-side rows even if no match
FULL OUTER JOIN	All rows from both tables, NULLs where no match	You want everything from both tables
CROSS JOIN	Every combination of rows (Cartesian product)	Generating all combinations (use carefully!)
SELF JOIN	A table joined to itself using aliases	Hierarchical data like managers → employees

```
-- LEFT JOIN example: show all students even if not enrolled
SELECT s.first_name, c.course_name
FROM students s
LEFT JOIN enrolments e ON s.student_id = e.student_id
LEFT JOIN courses c ON e.course_id = c.course_id;

-- SELF JOIN: employees and their managers
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

Useful Built-in Functions

Category	Function	Example	Result
String	UPPER(str)	UPPER('hello')	'HELLO'
String	LOWER(str)	LOWER('WORLD')	'world'
String	LENGTH(str)	LENGTH('abc')	3

Category	Function	Example	Result
String	CONCAT(a,b)	CONCAT('Hi',' ','there')	'Hi there'
String	SUBSTRING(s,pos,n)	SUBSTRING('database',1,4)	'data'
String	TRIM(str)	TRIM(' hello ')	'hello'
Math	ROUND(n,d)	ROUND(3.14159,2)	3.14
Math	ABS(n)	ABS(-42)	42
Math	MOD(a,b)	MOD(10,3)	1
Date	NOW()	NOW()	2025-01-15 10:30:00
Date	DATE(dt)	DATE(NOW())	2025-01-15
Date	DATEDIFF(a,b)	DATEDIFF('2025-12-31','2025-01-01')	364
Agg.	COUNT(col)	COUNT(student_id)	Integer
Agg.	SUM(col)	SUM(credits)	Numeric sum
Agg.	AVG(col)	AVG(grade_points)	Numeric avg
Agg.	MAX/MIN(col)	MAX(salary)	Max/min value

Chapter 6 — Indexes & Query Performance

What is an Index?

An **index** is a separate data structure (usually a B-Tree) that the database maintains to make lookups on a column dramatically faster — similar to the index at the back of a textbook. Without an index, every query must scan every single row (a *full table scan*).

```

-- Create a simple index on email (single column)
CREATE INDEX idx_student_email ON students (email);

-- Composite index (useful for queries filtering on both columns)
CREATE INDEX idx_name ON students (last_name, first_name);

-- Unique index (prevents duplicates AND speeds up lookups)
CREATE UNIQUE INDEX idx_unique_email ON students (email);

-- See all indexes on a table (MySQL)
SHOW INDEX FROM students;

-- Drop an index
DROP INDEX idx_student_email ON students; -- MySQL
DROP INDEX idx_student_email; -- PostgreSQL

```

Scenario	Index Helps?	Reason
WHERE email = 'x@y.com'	YES ■	Exact match on indexed column
WHERE last_name LIKE 'Sha%'	YES ■	Prefix search uses index
WHERE last_name LIKE '%arma'	NO ■	Leading wildcard bypasses index
ORDER BY indexed_column	YES ■	Index is already sorted
JOIN on Foreign Key column	YES ■	Index on FK speeds up join
High-cardinality column (many values)	YES ■	Index is most effective here
Low-cardinality (e.g., gender M/F)	NO ■	Faster to scan; too few distinct values

■ Do NOT add indexes to every column! Each index slows down INSERT/UPDATE/DELETE operations and uses disk space. Index only columns frequently used in WHERE, JOIN, or ORDER BY.

Chapter 7 — Transactions & Concurrency

A **transaction** is a sequence of one or more SQL operations treated as a single logical unit of work. Either all operations commit successfully, or none of them take effect (rollback).

```
-- Classic bank transfer example
BEGIN; -- or START TRANSACTION;

UPDATE accounts SET balance = balance - 5000 WHERE account_id = 101;
UPDATE accounts SET balance = balance + 5000 WHERE account_id = 202;

-- Check for errors before committing
COMMIT; -- Makes both changes permanent

-- OR if something went wrong:
ROLLBACK; -- Undoes EVERYTHING since BEGIN

-- SAVEPOINT: partial rollback within a transaction
BEGIN;
INSERT INTO orders (...) VALUES (...);
SAVEPOINT order_created;
INSERT INTO order_items (...) VALUES (...); -- This fails
ROLLBACK TO SAVEPOINT order_created; -- Undo only items insert
COMMIT; -- Commit the order itself
```

Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Performance
READ UNCOMMITTED	Possible	Possible	Possible	Fastest (dangerous)
READ COMMITTED	Safe	Possible	Possible	Fast
REPEATABLE READ	Safe	Safe	Possible	Medium (MySQL default)
SERIALIZABLE	Safe	Safe	Safe	Slowest (safest)

Chapter 8 — Security & User Management

```
-- Create a new database user
CREATE USER 'app_user'@'localhost' IDENTIFIED BY 'StrongP@ss123!';

-- Grant only necessary permissions (Principle of Least Privilege)
GRANT SELECT, INSERT, UPDATE ON school_db.* TO 'app_user'@'localhost';

-- Grant all on specific table only
GRANT ALL PRIVILEGES ON school_db.students TO 'app_user'@'localhost';

-- Revoke a permission
REVOKE DELETE ON school_db.* FROM 'app_user'@'localhost';

-- See what permissions a user has
SHOW GRANTS FOR 'app_user'@'localhost';

-- Remove a user entirely
DROP USER 'app_user'@'localhost';

-- Apply changes immediately
FLUSH PRIVILEGES;
```

Security Best Practices

- **Never use the root/admin account** for application connections. Always create dedicated users.
- **Use strong passwords:** minimum 12 characters with uppercase, lowercase, digits and symbols.
- **Principle of Least Privilege:** Grant only the permissions the application actually needs.
- **Encrypt sensitive data:** Never store plain-text passwords. Use bcrypt or Argon2.
- **Encrypt connections:** Always use SSL/TLS for remote database connections.
- **Disable remote root login:** Root should only connect from localhost.
- **Audit logs:** Enable query logging to detect suspicious activity.
- **Parameterised queries:** NEVER concatenate user input into SQL strings — prevents SQL Injection.
- **Regular updates:** Keep database software patched against known vulnerabilities.

■ **SQL Injection is the #1 database attack vector. ALWAYS use prepared statements / parameterised queries when accepting user input. Example (Python):**
`cursor.execute('SELECT * FROM users WHERE email = %s', (email,))`

Chapter 9 — Backup, Recovery & Maintenance

[illegible]

Backup Strategy (3-2-1 Rule)

■ 3 copies of data • 2 different media types • 1 off-site copy. Test your restores regularly — an untested backup is not a backup.

Backup Type	Description	Frequency	Storage Cost
Full Backup	Complete copy of entire database	Weekly	High
Incremental Backup	Only changes since last backup	Daily	Low
Differential Backup	Changes since last FULL backup	Daily	Medium

Backup Type	Description	Frequency	Storage Cost
Transaction Log	Log of every transaction (point-in-time)	Hourly	Low

Chapter 10 — Best Practices & Common Mistakes

Naming Conventions

Object	Convention	Good Example	Bad Example
Table	Lowercase, plural, snake_case	order_items	OrderItems, Orders_Tbl
Column	Lowercase, snake_case, descriptive	first_name	fn, firstName
Primary Key	tablename_id or id	user_id	pk, id1
Foreign Key	referenced_table_id	customer_id	cust, fk1
Index	idx_table_column(s)	idx_users_email	index1
View	vw_ prefix	vw_active_users	v1, MyView
Stored Proc	sp_ or usp_ prefix	sp_get_orders	proc1

Do's and Don'ts

DO ■	DON'T ■
Use transactions for multi-step writes	Run UPDATE/DELETE without WHERE
Index columns used in WHERE/JOIN/ORDER BY	Index every column blindly
Use VARCHAR with appropriate length limits	Use TEXT for everything
Validate & sanitise all user inputs	Concatenate user input into SQL
Use connection pooling in applications	Open a new DB connection per request
Document your schema with comments	Leave tables and columns undocumented
Use EXPLAIN to analyse slow queries	Guess why a query is slow
Use appropriate data types (INT, DATE, DECIMAL)	Store dates as VARCHAR
Set NOT NULL on required fields	Allow NULLs everywhere
Back up before any major schema change	Alter production schema without backup

Use READ-ONLY replicas for reporting queries

Run heavy analytics on production DB

Chapter 11 — SQL Quick Reference Cheatsheet

DDL

Create DB	<code>CREATE DATABASE dbname;</code>
Drop DB	<code>DROP DATABASE dbname;</code>
Create Table	<code>CREATE TABLE t (col TYPE CONSTRAINT, ...);</code>
Add Column	<code>ALTER TABLE t ADD COLUMN col TYPE;</code>
Drop Column	<code>ALTER TABLE t DROP COLUMN col;</code>
Rename Table	<code>RENAME TABLE old TO new; (MySQL)</code>
Drop Table	<code>DROP TABLE t;</code>
Truncate	<code>TRUNCATE TABLE t; -- Delete all rows, keep structure</code>

DML

Insert	<code>INSERT INTO t (c1,c2) VALUES (v1,v2);</code>
Multi-Insert	<code>INSERT INTO t (c1) VALUES (v1),(v2),(v3);</code>
Update	<code>UPDATE t SET c1=v1 WHERE condition;</code>
Delete	<code>DELETE FROM t WHERE condition;</code>
Upsert (MySQL)	<code>INSERT INTO t (...) VALUES (...) ON DUPLICATE KEY UPDATE ...;</code>

DQL

Select All	<code>SELECT * FROM t;</code>
Select Cols	<code>SELECT c1, c2 FROM t WHERE cond;</code>
Alias	<code>SELECT c1 AS alias FROM t;</code>
Distinct	<code>SELECT DISTINCT c1 FROM t;</code>
Order & Limit	<code>SELECT * FROM t ORDER BY c1 DESC LIMIT 10 OFFSET 5;</code>
Group & Having	<code>SELECT c1, COUNT(*) FROM t GROUP BY c1 HAVING COUNT(*)>1;</code>
Inner Join	<code>SELECT ... FROM a JOIN b ON a.id = b.a_id;</code>
Left Join	<code>SELECT ... FROM a LEFT JOIN b ON a.id = b.a_id;</code>

Subquery	<code>SELECT * FROM t WHERE id IN (SELECT id FROM t2);</code>
CASE	<code>SELECT CASE WHEN x>90 THEN 'A' ELSE 'B' END FROM t;</code>

Index & View

Create Index	<code>CREATE INDEX idx_name ON t(col);</code>
Unique Index	<code>CREATE UNIQUE INDEX idx ON t(col);</code>
Drop Index	<code>DROP INDEX idx ON t; (MySQL)</code>
Create View	<code>CREATE VIEW vname AS SELECT ... FROM t;</code>
Drop View	<code>DROP VIEW vname;</code>

Users & Perms

Create User	<code>CREATE USER 'u'@'host' IDENTIFIED BY 'pass';</code>
Grant	<code>GRANT SELECT, INSERT ON db.* TO 'u'@'host';</code>
Revoke	<code>REVOKE DELETE ON db.* FROM 'u'@'host';</code>
Drop User	<code>DROP USER 'u'@'host';</code>
Show Grants	<code>SHOW GRANTS FOR 'u'@'host';</code>

Chapter 12 — What to Learn Next

You now have a solid foundation in databases. Here is a roadmap of topics to explore as you grow from beginner to intermediate to advanced:

Beginner → Intermediate

- Master complex JOINS and subqueries
- Learn window functions: ROW_NUMBER(), RANK(), LAG(), LEAD()
- Understand NULL handling (IS NULL, COALESCE, NULLIF)
- Practice with a real project (e.g. a library or inventory system)
- Learn to use EXPLAIN / EXPLAIN ANALYZE to debug slow queries

Intermediate → Advanced

- Study stored procedures, triggers, and user-defined functions
- Deep dive into indexing strategies and query optimisation
- Learn database replication (master-slave, master-master)
- Explore partitioning large tables for performance
- Understand MVCC (Multi-Version Concurrency Control)

Advanced Topics

- Distributed databases: sharding, consistent hashing
- NewSQL databases: CockroachDB, TiDB, Spanner
- Data warehousing: star schema, OLAP vs OLTP
- NoSQL deep dives: MongoDB aggregation pipeline, Redis data structures
- Database-as-a-Service: AWS RDS, Google Cloud SQL, Azure Database

■ **Recommended Resources:** [SQLZoo.net](#), [Mode SQL Tutorial](#), [PostgreSQL official docs](#), [Stanford DB Course \(free online\)](#), ['Learning SQL' by Alan Beaulieu \(O'Reilly\)](#).